

This chapter discusses the I2C bus and shows the interfacing of the DS1307 real-time clock (RTC), an I2C chip. In Section 18.1, we describe the I2C bus and focus on I2C terminology and protocols. In Section 18.2, we describe the registers of AVR associated with I2C. In Section 18.3, we show how to write a simple program in Assembly and C to use the I2C features of AVR. In Section 18.4, we describe the DS1307 RTC's pin functions and show its interfacing and programming with the AVR. Advanced programming of the I2C (TWI) is discussed in Section 18.5.

SECTION 18.1: I2C BUS PROTOCOL

The IIC (Inter-Integrated Circuit) is a bus interface connection incorporated into many devices such as sensors, RTC, and EEPROM. The IIC is also referred to as I2C (I²C) or I square C in many technical literatures. In this section we examine the pins of the I2C bus and focus on I2C terminology and protocols.

I2C bus

The I2C bus was originally started by Philips, but in recent years has become a widely used standard adapted by many semiconductor chip companies. I2C is ideal for attaching low-speed peripherals to a motherboard or embedded system or anywhere that a reliable communication over a short distance is required. As we will see in this chapter, I2C provides a connection-oriented communication with acknowledge. I2C devices use only 2 pins for data transfer, instead of the 8 or more pins used in traditional buses. They are called SCL (Serial Clock), which synchronize the data transfer between two chips, and SDA (Serial Data). This reduction of communication pins reduces the package size and power consumption drastically, making them ideal for many applications in which space is a major concern. These two pins, SDA and SCK, make the I2C a 2-wire interface. In many application notes, including AVR datasheets, I2C is referred to as *Two-Wire Serial Interface (TWI)*. In this chapter we use I2C and TWI interchangeably.

I2C line electrical characteristics

I2C devices use only 2 bidirectional open-drain pins for data communication. To implement I2C, only a 4.7 kilohm pull-up resistor for each of bus lines is needed (see Figure 18-1). This implements a wired-AND, which is needed to implement I2C protocols. This means that if one or more devices pull the line to

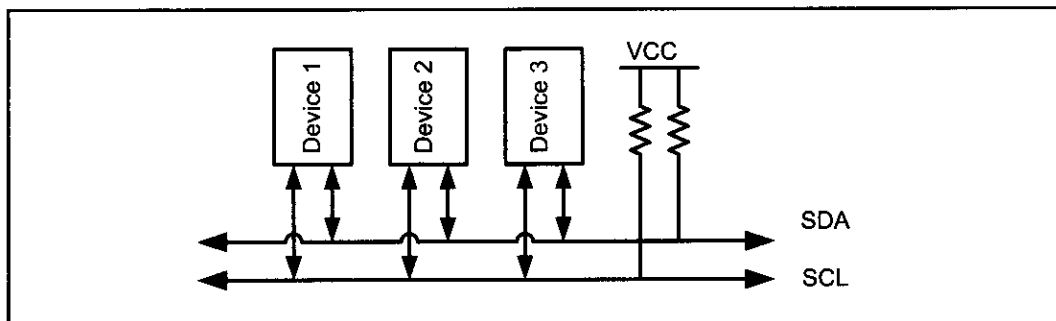


Figure 18-1. I2C Bus

low (zero) level, the line state is zero and the level of line will be 1 only if none of devices pull the line to low level.

I2C nodes

In the AVR up to 120 different devices can share an I2C bus. Each of these devices is called a *node*. In I2C terminology, each node can operate as either master or slave. Master is a device that generates the clock for the system; it also initiates and terminates a transmission. Slave is the node that receives the clock and is addressed by the master. In I2C, both master and slave can receive or transmit data, so there are four modes of operation. They are master transmitter, master receiver, slave transmitter, and slave receiver. Notice that each node can have more than one mode of operation at different times, but it has only one mode of operation at a given time. See Example 18-1.

Example 18-1

Give an example to show how a device (node) can have more than one mode of operation.

Solution:

If you connect the AVR to an EEPROM with I2C, the AVR does a master transmit operation to write to EEPROM. The AVR also does master receive operations to read from EEPROM. In the following sections, you will see that a node can do the operations of master and slave at different times.

Bit format

I2C is a synchronous serial protocol; each data bit transferred on the SDA line is synchronized by a high-to-low pulse of clock on the SCL line. According to I2C protocols the data line cannot change when the clock line is high; it can change only when the clock line is low. See Figure 18-2. The STOP and START conditions are the only exceptions to this rule.

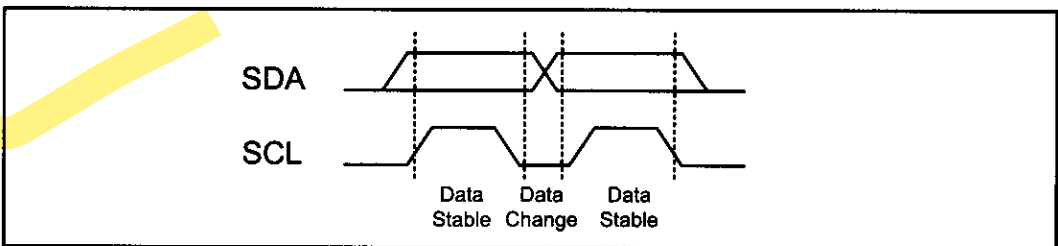


Figure 18-2. I2C Bit Format

START and STOP conditions

As we mentioned before, I2C is a connection-oriented communication protocol. This means that each transmission is initiated by a START condition and is terminated by a STOP condition. Remember that the START and STOP conditions are generated by the master.

STOP and START conditions must be distinguished from bits of address or data. That is why they do not obey the bit format rule that we mentioned before.

START and STOP conditions are generated by keeping the level of the SCL line high and then changing the level of the SDA line. The START condition is generated by a high-to-low change in the SDA line when SCL is high. The STOP condition is generated by a low-to-high change in the SDA line when SCL is low. See Figure 18-3.

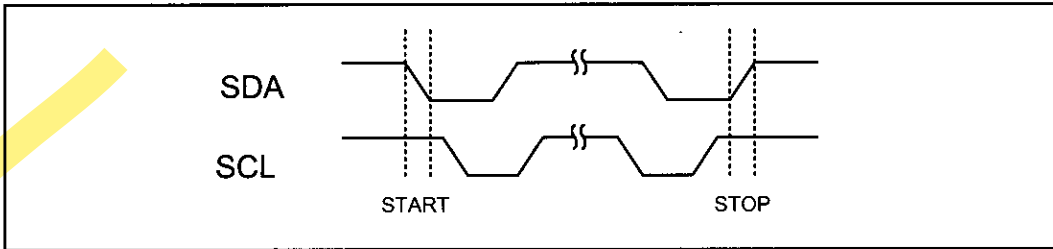


Figure 18-3. START and STOP Conditions

The bus is considered busy between each pair of START and STOP conditions, and no other master tries to take control of the bus when it is busy. If a master, which has the control of the bus, wishes to initiate a new transfer and does not want to release the bus before starting the new transfer, it issues a new START condition between a pair of START and STOP conditions. It is called the REPEATED START condition. See Figure 18-4.

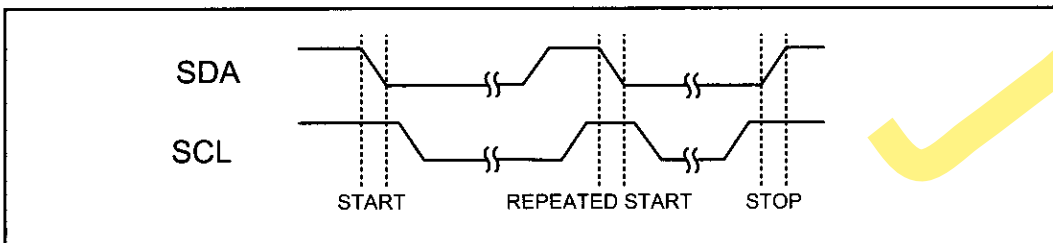


Figure 18-4. REPEATED START Condition

Example 18-2 shows why the REPEATED START condition is necessary.

Example 18-2

Give an example to show when a master must use the REPEATED START condition. What will happen if the master does not use it? [is easy read if time](#)

Solution:

If you connect two AVRs (AVR A and AVR B) and an EEPROM with I2C, and AVR A wants to display the addition of the contents of addresses 0x34 and 0x35 of EEPROM, it has to use the REPEATED START condition. Let's see what may happen if AVR A does not use the REPEATED START condition. AVR A transmits a START condition, reads the content of address 0x34 of EEPROM into R16, and transmits a STOP condition to release the bus. Before AVR A reads the contents of address 0x35 into R17, AVR B seizes the bus and changes the contents of addresses 0x34 and 0x35 of EEPROM. Then AVR A reads the content of address 0x35 into R17, adds it to R16, and displays the result on the LCD. The result on the LCD is neither the sum of the old values of addresses 0x34 and 0x35 nor the sum of the new values of addresses 0x34 and 0x35 of EEPROM!

Packet format in I2C

In I2C, each address or data to be transmitted must be framed in a packet. Each packet is 9 bits long. The first 8 bits are put on the SDA line by the transmitter, and the 9th bit is an acknowledge by the receiver or it may be NACK (not acknowledge). The clock is generated by the master, regardless of whether it is the transmitter or receiver. To get an acknowledge, the transmitter releases the SDA line during the ninth clock so that the receiver can pull the SDA line low to indicate an ACK. If the receiver doesn't pull the SDA line low, it is considered as NACK. See Figure 18-5.

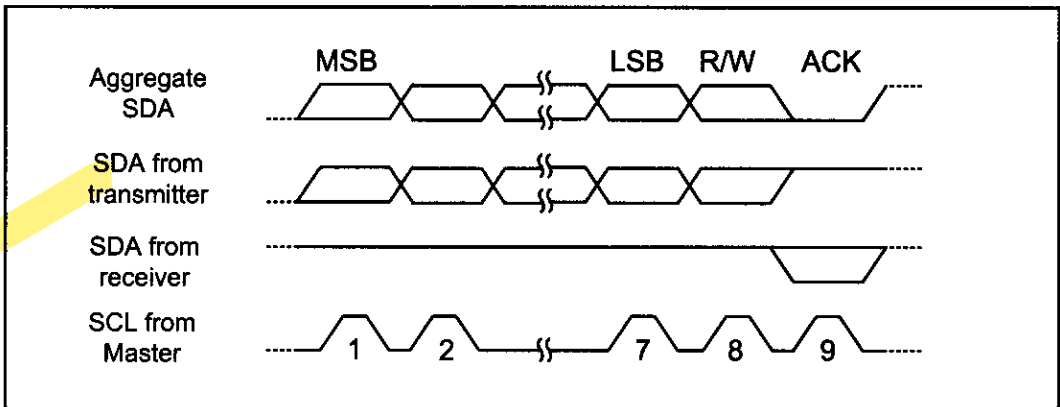


Figure 18-5. Packet Format in I2C

In I2C, each packet may contain either address or data. Also notice that START condition + address packet + one or more data packet + STOP condition together form a complete data transfer. Next we will study address and data packet formats and how to combine them to make a complete transmission.

Address packet format

Like any other packets, all address packets transmitted on the I2C bus are nine bits long. An address packet consists of seven address bits, one READ/WRITE control bit, and an acknowledge bit (see Figure 18-6).

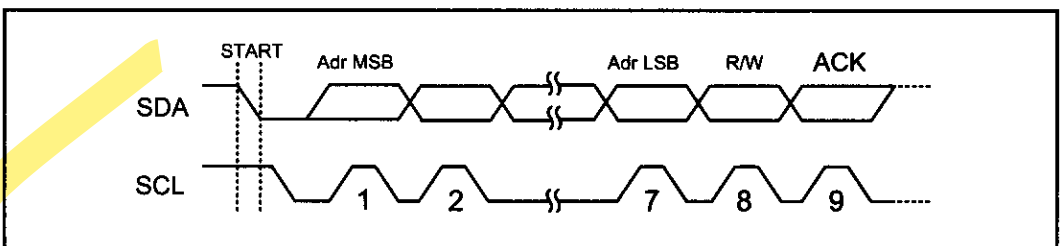


Figure 18-6. Address Packet Format in I2C

Address bits are used to address a specific slave device on the bus. The 7-bit address lets the master address a maximum of 128 slaves on the bus, although the address 0000 000 is reserved for general call and all addresses of the format 1111 xxx are reserved. That means 119 ($128 - 1 - 8$) devices can share an I2C bus. In the I2C bus the MSB of the address is transmitted first.

The eighth bit in the packet is the READ/WRITE control bit. If this bit is set, the master will read the next frame (Data) from the slave, otherwise, the mas-

ter will write the next frame (Data) on the bus to the slave. When a slave detects its address on the bus, it knows that it is being addressed and it should acknowledge in the ninth SCL (ACK) cycle by changing SDA to zero. If the addressed slave is not ready or for any reason does not want to service the master, it should leave the SDA line high in the ninth clock cycle. This is considered to be NACK. In case of NACK, the master can transmit a STOP condition to terminate the transmission, or a REPEATED START condition to initiate a new transmission.

Example 18-3 shows how a master says that it wants to write to a slave.

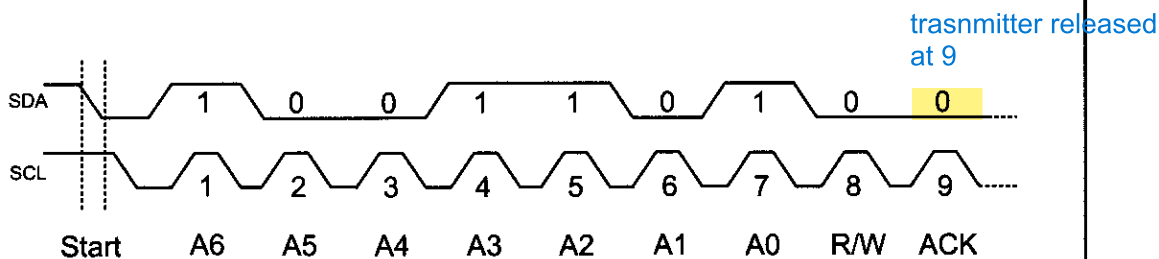
Example 18-3

Show how a master says that it wants to write to a slave with address 1001101.

Solution:

The following actions are performed by the master:

- (1) The master puts a high-to-low pulse on SDA, while SCL is high to generate a start bit condition to start the transmission.
- (2) The master transmits 10011010 into the bus. The first seven bits (1001101) indicates the slave address, and the eighth bit (0) indicates a Write operation and says that the master will write the next byte (data) into the slave.



An address packet consisting of a slave address and a READ is called SLA+R, while an address packet consisting of a slave address and a WRITE is called SLA+W.

As we mentioned before, address 0000 000 is reserved for general call. This means that when a master transmits address 0000 000, all slaves respond by changing the SDA line to zero and wait to receive the data byte. This is useful when a master wants to transmit the same data byte to all slaves in the system. Notice that the general call address cannot be used to read data from slaves because no more than one slave is able to write to the bus at a given time.

Data packet format

Like other packets, data packets are 9 bits long too. The first 8 bits are a byte of data to be transmitted, and the 9th bit is ACK. If the receiver has received the last byte of data and there is no more data to be received, or the receiver cannot receive or process more data, it will signal a NACK by leaving the SDA line high. In data packets, like address packets, MSB is transmitted first.

Combining address and data packets into a transmission

In I2C, normally, a transmission is started by a START condition, followed by an address packet (SLA + R/W), one or more data packets, and finished by a

STOP condition. Figure 18-7 shows a typical data transmission. Try to understand each element in the figure (see Example 18-4).

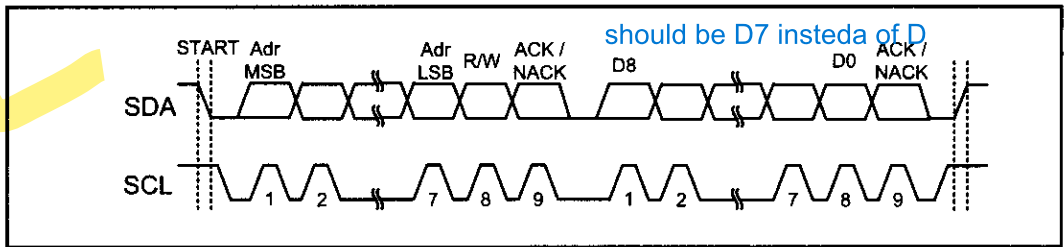


Figure 18-7. Typical Data Transmission

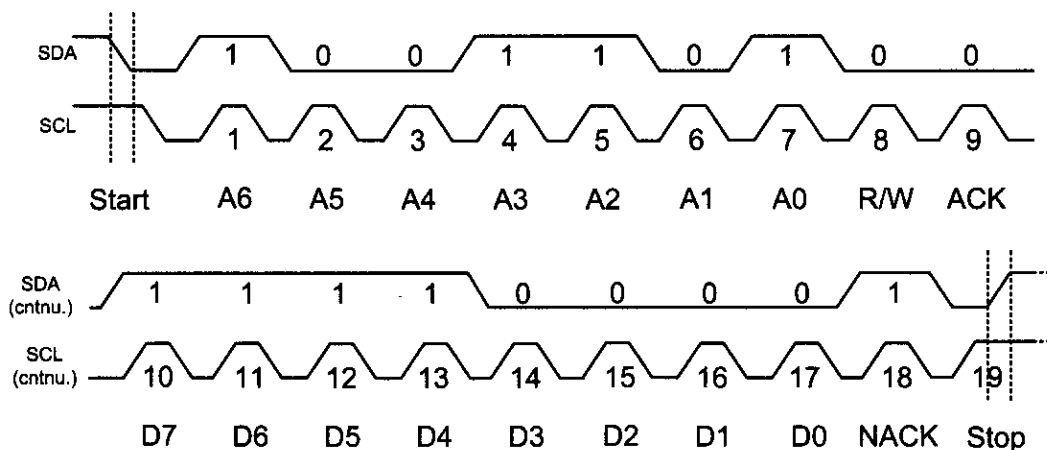
Example 18-4

Show how a master writes the value 11110000 to a slave with address 1001101.

Solution:

The following actions are performed by the master:

- (1) The master puts a high-to-low pulse on SDA while, SCL is high to generate a START condition to start the transmission.
- (2) The master transmits 10011010 into the bus. The first seven bits (1001101) indicate the slave address, and the eighth bit (0) indicates the Write operation stating that the master will write the next byte (data) into the slave.
- (3) The slave pulls the SDA line low to signal an ACK to say that it is ready to receive the data byte.
- (4) After receiving the ACK, the master will transmit the data byte (11110000) on the SDA line (MSB first).
- (5) When the slave device receives the data it leaves the SDA line high to signal NACK. This informs the master that the slave received the last data byte and does not need any more data.
- (6) After receiving the NACK, the master will know that no more data should be transmitted. The master changes the SDA line when the SCL line is high to transmit a STOP condition and then releases the bus.



Clock stretching

One of the features of the I²C protocol is clock stretching. It is a kind of flow control. If an addressed slave device is not ready to process more data it will stretch the clock by holding the clock line (SCL) low after receiving (or sending) a bit of data. Thus the master will not be able to raise the clock line (because devices are wire-ANDed) and will wait until the slave releases the SCL line to show it is ready to transfer the next bit. See Figure 18-8.

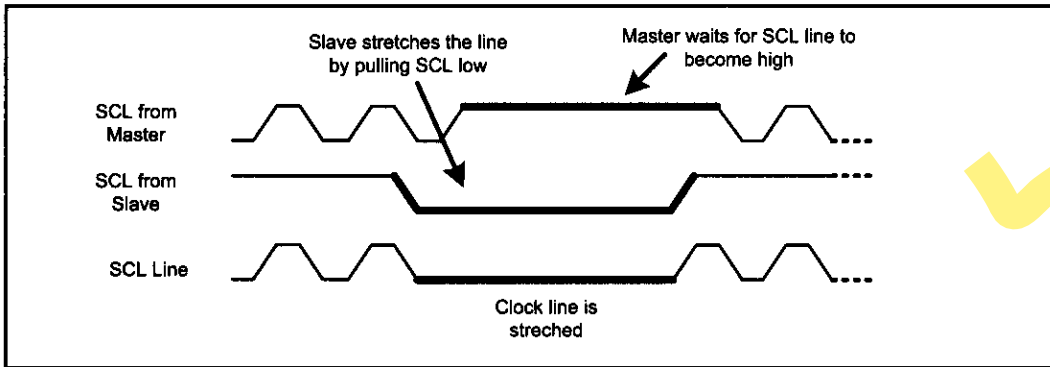


Figure 18-8. Clock Stretching

Arbitration

I²C protocol supports a multimaster bus system. This doesn't mean that more than one master can use the bus at the same time. Rather, each master waits for the current transmission to finish and then starts to use the bus. But it is possible that two or more masters initiate a transmission at about the same time. In this case the arbitration happens.

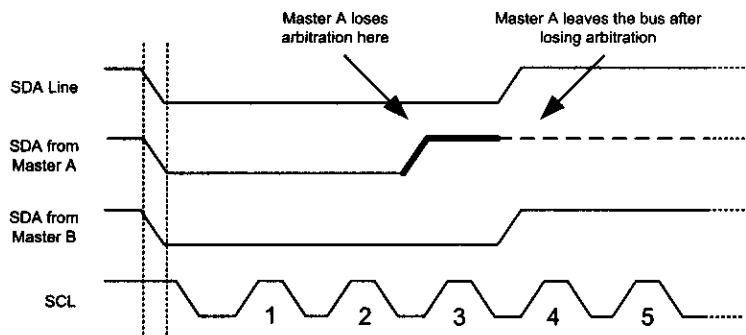
Each transmitter has to check the level of the bus and compare it with the level it expects; if it doesn't match, that transmitter has lost the arbitration, and will switch to slave mode. In the case of arbitration, the winning master will continue its job. Notice that neither the bus is corrupted nor the data is lost. See Example 18-5.

Example 18-5

Two masters, A and B, start at about the same time. What happens if master A wants to write to slave 0010 000 and master B wants to write to slave 0001 111?

Solution:

Master A will lose the arbitration in the third clock because the SDA line is different from the output of master A at the third clock. Master A switches to slave mode and leaves the bus after losing the arbitration.



Multibyte burst write

Burst mode writing is an effective means of loading consecutive locations. It is supported in I2C, SPI, and many other serial protocols. In burst mode, we provide the address of the first location, followed by the data for that location. From then on, consecutive bytes are written to consecutive memory locations. In this mode, the I2C device internally increments the address location as long as the STOP condition is not detected. The following steps are used to send (write) multiple bytes of data in burst mode for I2C devices.

1. Generate a START condition.
2. Transmit the slave address followed by zero (for write).
3. Transmit the address of the first location.
4. Transmit the data for the first location and from then on, simply provide consecutive bytes of data to be placed in consecutive memory locations.
5. Generate a STOP condition.

Figure 18-9 shows how to write 0x01, 0x02, and 0x03 to three consecutive locations starting from location 00001111 of slave 1111000.

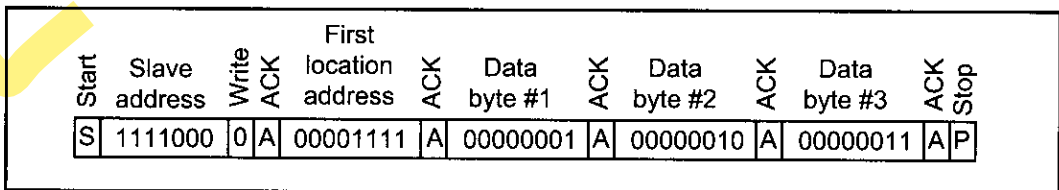


Figure 18-9. Multibyte Burst Write

start + start address + datas

Multibyte burst read

Burst mode reading is an effective way of bringing out the contents of consecutive locations. In burst mode, we provide the address of the first location only. From then on, contents are brought out from consecutive memory locations. In this mode, the I2C device internally increments the address location as long as the STOP condition is not detected. The following steps are used to get (read) multiple bytes of data using burst mode for I2C devices.

1. Generate a START condition.
2. Transmit the slave address followed by zero (for address write).
3. Transmit the address of the first location.
4. Generate a START (REPEATED START) condition.
5. Transmit the slave address followed by one (for read).
6. Read the data from the first location and from then on, bring contents out from consecutive memory locations.
7. Generate a STOP condition.

Figure 18-10 shows how to read three consecutive locations starting from location 00001111 of slave number 1111000.

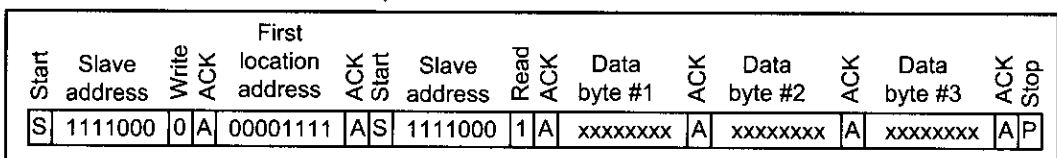


Figure 18-10. Multibyte Burst Read

Review Questions

1. True or false. I2C protocol is ideal for short distances.
2. How many bits are there in a frame? Which bit is for acknowledge?
3. True or false. START and STOP conditions are generated when the SDA is high.
4. What is the name of the flow control method in the I2C protocol?
5. What is the recommended value for the pull-up resistors in the I2C protocol?
6. True or false. After the arbitration of two masters, both of them must start transmission from the beginning.

SECTION 18.2: TWI (I2C) IN THE AVR

In many applications, including AVR datasheet, I2C is referred to as *Two-wire Serial Interface (TWI)*. From now on, in this book we use TWI to conform with the AVR data sheets. In this section we discuss the TWI module and registers of the AVR. Then we show how to program the AVR to address a slave device and send or receive data using TWI. The TWI module in the AVR is composed of four submodules: bit rate generation unit, bus interface unit, address match unit, and control unit. Figure 18-11 shows the TWI module. All registers drawn with a thick line are accessible through the AVR data bus.

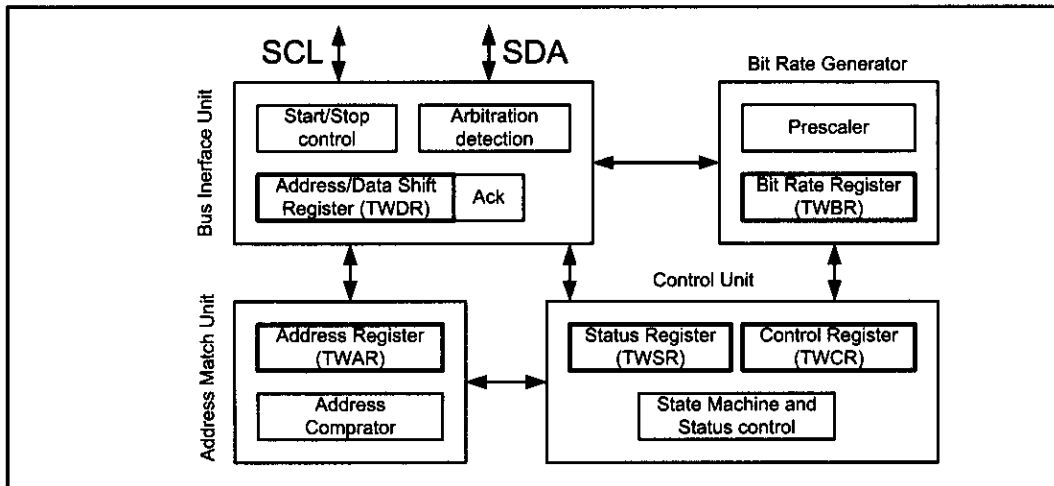


Figure 18-11. TWI (I2C) in AVR

The bit rate generation unit controls the frequency of the system clock (SCL) when operating in a master mode. The bus interface unit detects and generates START, REPEATED START and STOP conditions. It also detects arbitration, controls sending or receiving ACK, and also transfers packets of data or address. The address match unit compares the received address byte with the 7-bit address in TWI address register and informs the control unit upon an address match. The control unit controls the TWI module and generates responses according to settings in the TWI control register. It also sets the contents of the status register according to current state.

In the AVR microcontroller, five major registers are associated with the TWI. They are TWBR (TWI Bit rate Register), TWCR (TWI Control Register), TWSR (TWI Status Register), TWAR (TWI Address Register), and TWDR (TWI

Data Register). Next, we will focus on registers related to TWI and study each bit of them in detail.

TWI Bit Rate Register (TWBR)

The following figure shows the TWBR register and its bits.

TWBR7	TWBR6	TWBR5	TWBR4	TWBR3	TWBR2	TWBR1	TWBR0
-------	-------	-------	-------	-------	-------	-------	-------

TWBR selects the division factor to control the SCL clock frequency in master mode. The SCL frequency is controlled by settings in the TWBR and the prescaler bits in the TWSR (TWI status register is discussed next). The following equation demonstrates the relation between SCL frequency, TWBR, and TWPS bits in TWI status register:

$$\text{SCL frequency} = \frac{\text{CPU Clock frequency}}{16 + 2 (\text{TWBR}) \times 4^{\text{TWPS}}}$$

Notice that the value of TWBR should be 10 or higher if the TWI operates in master mode. Example 18-6 shows how the frequency of SCL is calculated.

Example 18-6

Calculate the SCL frequency if the value of TWPS bits in TWSR is 01 (1 Dec) and the value of TWBR is 00100110 (38 Dec). Assume that CPU clock frequency is 8 MHz.

Solution:

The SCL frequency will be: $8 \text{ MHz} / ((16 + 2 (38) \times 4) = 25 \text{ kHz}$

TWI Status Register (TWSR)

As you see in Figure 18-12, five bits of TWSR are dedicated to show the status of the TWI logic and bus. Notice that if you read TWSR, you will read both the status bits and the prescaler value. To check the status bits, you should mask the two LSB bits (prescaler values) to zero. In this book we do not list all of the status codes and their meanings, but we will cover some of more common ones. To see the complete list of status register codes, you should refer to the data sheet of the chip. Next we will see how to use these bits when we want to program the AVR to use the TWI module.

TWS7	TWS6	TWS5	TWS4	TWS3	-	TWPS1	TWPS0
------	------	------	------	------	---	-------	-------

Bits 7..3 – TWS: TWI Status

These five bits show the status of the TWI control and bus.

Bits 1..0 – TWPS: TWI Prescaler Bits

These bits control the bit rate prescaler.

Figure 18-12. TWSR: TWI Status Register

TWI Control Register (TWCR)

TWCR controls the operation of the TWI. In Figure 18-13 you see each bit of TWCR and a short description of it. Here we will describe some of these bits in more detail.

TWINT	TWEA	TWSTA	TWSTO	TWWC	TWEN	-	TWIE
Bit 7 – TWINT: TWI Interrupt This bit is set by hardware when the TWI module has finished its current job. If the TWI and general interrupt are enabled, changing TWINT to one will cause the MCU to jump to the TWI interrupt vector. Clearing this flag starts the operation of the TWI. TWINT must be cleared by software.							
Bit 6 – TWEA: TWI Enable Acknowledge Making this bit HIGH will enable the generation of ACK when needed in slave or receiver mode.							
Bit 5 – TWSTA: TWI START condition Bit Making this bit HIGH will generate a START condition if the bus is free; otherwise, the TWI module waits for the bus to become free and then generates a START condition							
Bit 4 – TWSTO: TWI STOP condition bit In master mode, making this bit HIGH causes the TWI to generate a STOP condition. This bit is cleared by hardware when the STOP condition is transmitted.							
Bit 3 – TWWC: TWI Write Collision Flag This bit is set HIGH when we attempt to access the TWI Data Register when TWINT is low. This flag is cleared by writing to the TWDR register when TWINT is high.							
Bit 2 – TWEN: TWI Enable Making this bit HIGH enables the TWI module.							
Bit 0 – TWIE: TWI Interrupt Enable Making this bit HIGH enables the TWI interrupt if the general interrupt is enabled.							

Figure 18-13. TWCR: TWI Control Register

TWI Interrupt (TWINT) flag

When the TWI hardware finishes its job, it sets the TWINT bit to one. If the TWI and general interrupts are enabled, changing TWINT to HIGH will cause the MCU to jump to the TWI interrupt vector. When the TWINT bit is set, the TWI module “stretches” the SCL line to provide enough time for software to do specified jobs. When the software finishes its job, it must clear the TWINT bit to resume the operation of the TWI module. Notice that all accesses to the TWI address, status, and data registers must be complete before clearing this flag. If you try to write to the TWI Data Register when TWINT is low, a collision will happen and the TWI collision flag (TWWC) will be set to HIGH by hardware. Software can monitor (poll) the TWI bit to know when the TWI module finishes its job and is ready for a new command.

TWI Enable Acknowledge (TWEA) bit

Making this bit HIGH will enable the generation of the ACK bit if any of the following conditions are met:

1. The TWI Address Match module detects that the TWI module is addressed by receiving its own slave address from the bus.
2. A general call has been received while the TWGCE bit in the TWAR is set to one (to enable accepting of global calls).
3. A data byte has been received in each of the receiving modes, master receiver or slave receiver mode.

If you clear the TWEA bit to zero, the device will not generate ACK and will be virtually disconnected from the TWI bus.

TWI Start bit and TWI Stop bit (TWSTA and TWSTO)

To generate START or STOP conditions, you have to set the TWSTA or TWSTO bit to one respectively and then clear the TWINT flag to zero by writing a one to it.

TWI Data Register (TWDR)

In Receive mode, the last received byte will be in the TWDR, and in Transmit mode, you should write the next byte into TWDR to be transmitted. As we mentioned before, you can access the TWDR only when the TWIE is set to one otherwise collision happens. This means the Data Register cannot be initialized by the user before the first interrupt occurs.

TWI Address Register (TWAR)

TWAR contains the 7-bit slave address to which the TWI will respond when working as slave. The eighth bit (LSB) of TWAR is TWGCE (TWI General Call Recognition Enable). It controls recognition of general call address (00). If this bit is set to one, receiving of a general call address will cause an interrupt request.

Review Questions

1. True or false. The AVR has an internal TWI module.
2. What are the TWI registers in AVR?
3. How do we generate START or STOP conditions in the AVR?
4. True or false. After reading status register we should mask the 2 MSB bits.
5. Which bit is polled to know if the TWI is ready now?
6. True or false. We can write to TWDR when the TWI module is busy.
7. Which bit controls the generation of ACK?

SECTION 18.3: AVR TWI PROGRAMMING IN ASSEMBLY AND C

In this section we discuss TWI programming in Assembly and C. Here we will focus on the simplest form of TWI programming without checking the status register. In most applications, if you are not dealing with critical systems and there is not more than one master on a single bus, you can use this method. If you want to deal with multimaster or critical designs you must check the value of the status flag. TWI programming with checking the value of the status flag is discussed in a later section.

In I2C protocol, a device can be either master or slave. In this section we will discuss the steps of programming in each mode.

Programming of the AVR TWI in master operating mode

To work in master operating mode, we must be able to initialize the TWI, transmit a START condition, send or receive data, and transmit a STOP condition. Next we will discuss each one in more detail.

Initialization

To initialize the TWI module to operate in master operating mode, we should do the following steps:

1. Set the TWI module clock frequency by setting the values of the TWBR register and the TWPS bits in the TWSR register.
2. Enable the TWI module by setting the TWEN bit in the TWCR register to one.

Transmit START condition

To start data transfer in master operating mode, we must transmit a START condition. This is done by setting the TWEN, TWSTA, and TWINT bits of TWCR to one. Setting the TWEN bit to one enables the TWI module. Setting the TWSTA bit to one tells the TWI to initiate a START condition when the bus is free, and setting the TWINT bit to one clears the interrupt flag to initiate operation of the TWI module to transmit the START condition. Then we should poll the TWINT flag in the TWCR register to see whether the START condition transmitted completely.

Send data

To send a byte of data, after transmitting the START condition, we should do the following steps:

1. Copy the data byte to the TWDR.
2. Set the TWEN and TWINT bits of the TWCR register to one to start sending the byte.
3. Poll the TWINT flag in the TWCR register to see whether the byte transmitted completely.

Notice that right after the START condition, we should transmit SLA + W (Slave Address + Write) or SLA + R (Slave Address + Read). As we mentioned in the first section, right after sending SLA+W we should write to the slave, and right after sending SLA+R we should read from it. To transmit SLA + R, SLA + W, and to write a byte of data to a slave we use a function called I2C_WWRITE.

Receive data

To receive a byte of data, after transmitting of SLA + R, we should do the following steps:

1. Set the TWEN and TWINT bits of the TWCR register to one to start receiving a byte. Notice that if you want to return ACK after receiving data you should also set the TWEA bit of the TWCR register to one.
2. Poll the TWINT flag in the TWCR register to see whether a byte has been received completely.
3. Copy the received byte from the TWDR to another register to save it.

Transmit STOP condition

To stop data transfer, we must transmit a STOP condition. This is done by setting the TWEN, TWSTO, and TWINT bits of the TWCR register to one. Notice that we cannot poll the TWINT flag after transmitting the STOP condition.

Program 18-1 shows how a master writes 11110000 to a slave with address 1101000.

```
;Tested OK-ok
.INCLUDE "M32DEF.INC"

    LDI    R21,HIGH(RAMEND)        ;set up stack
    OUT    SPH,R21
    LDI    R21,LOW(RAMEND)
    OUT    SPL,R21

    CALL   I2C_INIT                ;initialize TWI module
    CALL   I2C_START               ;transmit START condition
    LDI    R27, 0b11010000        ;SLA(1001100) + W (0)
    CALL   I2C_WRITE               ;write R27 to I2C bus
    LDI    R27, 0b11110000        ;data to be transmitted
    CALL   I2C_WRITE               ;write R27 to I2C bus
    CALL   I2C_STOP                ;transmit STOP condition

HERE: RJMP  HERE                  ;wait here forever

;*****
I2C_INIT:
    LDI    R21, 0
    OUT    TWSR,R21                ;set prescaler bits to zero
    LDI    R21, 0x47
    OUT    TWBR,R21                ;move 0x47 into R21
    OUT    TWBR,R21                ;set clock freq. to 50k (8 MHz XTAL)
    LDI    R21, (1<<TWEN)
    OUT    TWCR,R21                ;move 0x04 into R21
    RET                                ;enable the TWI

;*****
I2C_START:
    LDI    R21, (1<<TWINT) | (1<<TWSTA) | (1<<TWEN)
    OUT    TWCR,R21                ;transmit a START condition
```

Program 18-1: Writing a Byte in Master Mode (continued on next page)


```

WAIT1:
    IN    R21, TWCR                ;read control register into R21
    SBRS  R21, TWINT               ;skip next line if TWINT is 1
    RJMP  WAIT1                   ;jump to WAIT1 if TWINT is 1
    RET

;*****
I2C_WRITE:
    OUT   TWDR, R27                ;move the byte into TWDR
    LDI   R21, (1<<TWINT) | (1<<TWEN)
    OUT   TWCR, R21               ;configure TWCR to send TWDR
WAIT3:
    IN    R21, TWCR                ;read control register into R21
    SBRS  R21, TWINT               ;skip next line if TWINT is 1
    RJMP  WAIT3                   ;jump to WAIT3 if TWINT is 1
    RET

;*****
I2C_STOP:
    LDI   R21, (1<<TWINT) | (1<<TWSTO) | (1<<TWEN)
    OUT   TWCR, R21               ;transmit STOP condition
    RET

```

Program 18-1: Writing a Byte in Master Mode *(continued from previous page)*

Program 18-2 shows how to read a byte from a slave with address 1001100 and displays the result on Port A.

```

;Tested OK- ok
.INCLUDE "M32DEF.INC"

    LDI   R21,HIGH(RAMEND)         ;set up stack
    OUT   SPH,R21
    LDI   R21,LOW(RAMEND)
    OUT   SPL,R21

    LDI   R21,$FF                 ;move $FF to R21
    OUT   DDRA, R21               ;Port A is output

    CALL  I2C_INIT                ;initialize TWI module
    CALL  I2C_START               ;transmit START condition
    LDI   R27, 0b11010001         ;SLA(1001100) + R (1)
    CALL  I2C_WRITE               ;write R27 to I2C bus
    CALL  I2C_READ                ;write R27 to I2C bus
    OUT   PORTA,R27               ;show received data on Port A
    CALL  I2C_STOP                ;transmit STOP condition

HERE:
    RJMP  HERE                   ;wait here forever

```

Program 18-2: Reading a Byte in Master Mode *(continued on next page)*

```

;*****
I2C_INIT:
    LDI    R21, 0
    OUT    TWSR,R21                ;set prescaler bits to zero
    LDI    R21, $47                ;move $47 into R21
    OUT    TWBR,R21                ;SCL freq. is 50k for 8MHz XTAL
    LDI    R21, (1<<TWEN)          ;move 0x04 into R21
    OUT    TWCR,R21                ;enable the TWI
    RET

;*****
I2C_START:
    LDI    R21, (1<<TWINT)|(1<<TWSTA)|(1<<TWEN)
    OUT    TWCR,R21                ;transmit a START condition
WAIT1:
    IN     R21, TWCR                ;read control register into R21
    SBRS   R21, TWINT               ;skip next line if TWINT is 1
    RJMP   WAIT1                   ;jump to WAIT1 if TWINT is 1
    RET

;*****
I2C_READ:
    LDI    R21, (1<<TWINT)|(1<<TWEN)
    OUT    TWCR, R21
WAIT2:
    IN     R21, TWCR                ;read control register into R21
    SBRS   R21, TWINT               ;skip next line if TWINT is 1
    RJMP   WAIT2                   ;jump to WAIT2 if TWINT is 0
    IN     R27, TWDR                ;read received data into R21
    RET

;*****
I2C_WRITE:
    OUT    TWDR, R27                ;move the byte into TWDR
    LDI    R21, (1<<TWINT)|(1<<TWEN)
    OUT    TWCR, R21                ;configure TWCR to send TWDR
WAIT3:
    IN     R21, TWCR                ;read control register into R21
    SBRS   R21, TWINT               ;skip next line if TWINT is 1
    RJMP   WAIT3                   ;jump to WAIT3 if TWINT is 1
    RET

;*****
I2C_STOP:
    LDI    R21, (1<<TWINT)|(1<<TWSTO)|(1<<TWEN)
    OUT    TWCR, R21                ;transmit STOP condition
    RET

```

Program 18-2: Reading a Byte in Master Mode (continued from previous page)

C programming of the AVR TWI in master operating mode

Program 18-3 shows how a master writes 11110000 to a slave with address 1101000. This program is the C version of Program 18-1.

Program 18-4 shows how a master reads from a slave with address 1101000 and displays the result on Port A. This program is the C version of Program 18-2.

```
#include <avr/io.h>

void i2c_write(unsigned char data)
{
    TWDR = data ;
    TWCR = (1<< TWINT) | (1<<TWEN);
    while ((TWCR & (1 <<TWINT)) == 0);
}

/*****

void i2c_start(void)
{
    TWCR = (1 << TWINT) | (1 << TWSTA) | (1 << TWEN);
    while ((TWCR & (1 << TWINT)) == 0);
}

/*****

void i2c_stop()
{
    TWCR = (1<< TWINT) | (1<<TWEN) | (1<<TWSTO);
}

/*****

void i2c_init(void)
{
    TWSR=0x00;           //set prescaler bits to zero
    TWBR=0x47;           //SCL frequency is 50K for XTAL = 8M
    TWCR=0x04;           //enable the TWI module
}

/*****

int main (void)
{
    i2c_init();
    i2c_start();         //transmit START condition
    i2c_write(0b11010000); //transmit SLA + W(0)
    i2c_write(0b11110000); //transmit data
    i2c_stop();          //transmit STOP condition
    while(1);            //stay here forever
    return 0 ;
}
```

Program 18-3: Writing a Byte in Master Mode in C

```

#include <avr/io.h>

void i2c_init(void)
{
    TWSR=0x00;           //set prescaler bits to zero
    TWBR=0x47;           //SCL Frequency is 50K for XTAL=8M
    TWCR=0x04;           //enable the TWI module
}

//*****
void i2c_start(void)
{
    TWCR = (1 << TWINT) | (1 << TWSTA) | (1 << TWEN);
    while ((TWCR & (1 << TWINT)) == 0);
}

//*****
void i2c_write(unsigned char data)
{
    TWDR = data ;
    TWCR = (1<< TWINT)|(1<<TWEN);
    while ((TWCR & (1 <<TWINT)) == 0);
}

//*****
unsigned char i2c_read(unsigned char isLast)
{
    if (isLast == 0)      //if want to read more than 1 byte
        TWCR = (1<< TWINT)|(1<<TWEN)|(1<<TWEA);
    else                  //if want to read only one byte
        TWCR = (1<< TWINT)|(1<<TWEN);
    while ((TWCR & (1 <<TWINT)) == 0);
    return TWDR ;
}

//*****
void i2c_stop()
{
    TWCR = (1<< TWINT)|(1<<TWEN)|(1<<TWSTO);
}

//*****
int main (void)
{
    unsigned char i = 0 ;
    DDRA = 0xFF;           //Port A is output
    i2c_init();            //initialize TWI for master mode
    i2c_start();           //transmit START condition
    i2c_write(0b11010001); //transmit SLA + R(1)
    i=i2c_read(1);         //read only one byte of data
    PORTA= i;              //show the byte on Port A
    i2c_stop();            //transmit STOP condition
    while(1);              //stay here forever
    return 0 ;
}

```

Program 18-4: Reading a Byte in Master Mode in C